

Proving the Program You Run

Mutable borrows inside a dependent type theory, by observation and closing off

Abstract. Verified systems code today is written twice: once as the efficient imperative program that runs, and once as a pure model that the proofs are about, joined by a refinement proof or a translation. We present Ochr, a core calculus in which the two are the same text. Ochr combines Rust-style mutable borrows with a Lean-style dependent type theory, and its definitional equality unfolds in-place code exactly as Lean unfolds pure code. Three ideas make this work. First, an *observational* equality `Id A t u` compares two computations by what they return and what they leave behind in the places they may write; it is not a new primitive but computes to ordinary equality between observations. Second, a call whose body is stuck on an unknown value is *closed off*: its result, and the final content of every place it borrowed, become *sealed programs*, closed source programs that own their state. Sealed programs are the backward functions of Aeneas, but written in the source language, so the programmer never sees a second language. Third, induction hypotheses are typed in the caller’s environment, where the borrow structure around the recursive argument performs the congruence step that a pure proof would write by hand. Proofs about in-place functions become plain structural recursion, frequently shorter than the corresponding proofs about pure functions. We give the calculus, its worked examples (including a function that returns a borrow into its argument), a model in Lean’s type theory in which Aeneas’s backward functions reappear and which establishes consistency, and an executable Lean implementation of the checker.

1 Introduction

Here is an addition on Peano numerals that allocates nothing. It walks down the successors of `*x` until it reaches the final `Z` and moves `y` into that spot:

```
AddM(x : &Nat, y : Nat) : Unit by x :=  
  match *x { Z => *x := y | S p => AddM(&p, y) }
```

`&Nat` is a mutable borrow in the sense of Rust: a unique, temporary right to read and write a `Nat` owned by someone else. In the successor case the pattern variable `p` names the predecessor field in place, and `&p` reborrows it for the recursive call. Here is a theorem about `AddM`, stated and proved in the same language:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () by x :=  
  match *x { Z => refl | S p => AddMZero(&p) }
```

The statement says that running `AddM(x, 0)` is indistinguishable from doing nothing: both return `()`, and both leave the number behind `x` as it was. The proof is structural recursion and nothing else. There is no pure model of `AddM`, no refinement relation, no loop invariant, no separation-logic assertion and no translation into another language. The checker accepts the recursive call as a proof of the successor case by running both sides of the equation on a symbolic input and comparing what they leave behind.

This paper is about the type theory that makes that possible.

1.1 The two-language problem

Verified software is usually built in one of two ways. In the first, everything is written in a pure proof assistant such as Lean or Coq, and performance comes from the compiler’s ability to turn functional updates into in-place ones where it can prove the old value is dead. In the second, the efficient program is written in an imperative language, and a separate pure specification is related to it by a refinement proof: Verus [12] and Creusot [8] check Rust against specifications discharged by SMT, VeriFast checks C against separation-logic contracts, and Aeneas [10] translates safe Rust into a pure Lean program about which the user then proves theorems.

Both routes keep two artifacts. In the second route this is visible: there is a program and there is a specification, and the proof is about their relationship. In the first route it is hidden but present: the program the proof is about is not the program that runs, and the gap is bridged by an optimisation the programmer cannot state or rely on.

Aeneas comes closest to closing the gap. Its insight is that Rust’s ownership discipline makes mutation equivalent to state passing: a function taking $x : \&\text{mut } T$ is a pure function returning, besides its result, a *backward function* that computes the final value of $*x$. Aeneas performs this translation once, eagerly, for a whole program, and the user proves theorems about the translated Lean code, where the backward functions appear as ordinary definitions. The user therefore works in two languages at once, and must understand how one maps onto the other. We show that this translation can be performed **inside the type checker, lazily, as definitional equality, and in the source language itself.**

1.2 Three ideas

Observational equality. The type $\text{Id } A \ t \ u$ compares two computations t and u of result type A , run from the current state. It holds when they return equal values and leave equal contents in every place they may write. Id is not a new primitive with its own introduction and elimination rules. At a given environment it computes to an ordinary propositional equality between two *observations*, the tuples of result and final contents, and that equality computes further by the structure of the data in the manner of observational type theory [4]. The places a computation may write are read off its syntax, so they are an output of the computation like its result. This is the sense in which Ochr has no pure/impure divide: any program may appear in a type, and what it does to its environment is part of what the type talks about.

Closing off. Definitional equality in Ochr is normalisation by a deterministic symbolic machine: two terms are equal when the machine sends them to the same normal form. On symbolic inputs the machine gets stuck, and a dependent type theory needs a name, a *neutral term*, for every stuck computation. A stuck pure computation names itself: Lean’s $\text{Nat.add } n \ 0$ is its own normal form. A stuck effectful call does not, because part of what it produces lives in the places it borrowed. We *close off* such a call: its result and the final content of each borrowed place are replaced by *sealed programs*, closed source programs that own their state. For AddM on an unknown σ , the borrowed place receives $[\text{let } c = \sigma; \text{AddM}(\&c, \ 0); \ c]$, which is the pure wrapper a programmer would write anyway. When a later case split refines σ , the sealed program runs again. Sealed programs are exactly Aeneas’s backward functions, but they are written in the source language and generated on demand, so the programmer never writes or reads a second language. A function that returns a borrow is closed off into a sealed program with a hole awaiting the final value of the returned borrow; this is Aeneas’s region abstraction, in the same form.

The environment does the congruence. When the checker types the recursive call $\text{AddMZero}(\&p)$, it evaluates the callee’s statement in the caller’s environment, where $\&p$ points into the predecessor field of the caller’s number. The observation therefore sees the successor that surrounds the recursive call, and the induction hypothesis arrives already wrapped in S : it is literally the goal. The step $\text{cong } S \text{ ih}$ that a pure proof writes by hand has been performed by the borrow structure. Soundness rests on a frame property that Rust’s ownership discipline provides: a call can affect only what it is passed.

1.3 Why this is not impossible

Pédrot and Tabareau’s fire triangle [20] shows that a type theory cannot have substitution of arbitrary terms, dependent elimination and observable effects at once without becoming inconsistent. Ochr is call-by-value, keeps dependent elimination, and restricts what may be substituted into a type: a type only ever contains values. A computation mentioned in a type is run on a private copy of the environment, and only its observation, a value, enters the type. Ownership is what makes this restriction cheap. A closed Ochr program can mutate only places it created, so every closed program behaves like a value, and the observation of an open program is a value of a known shape.

1.4 Contributions

- A core calculus, *Ochr*, combining mutable borrows with a dependent type theory with a universe of proof-irrelevant propositions, whose definitional equality unfolds imperative code (Section 3, Section 4).
- *Closing off*, which gives every stuck effectful call a neutral form made of sealed source programs, including calls that return borrows (Section 4).
- An observational equality between computations that is derived rather than primitive (Section 5).
- Worked examples, among them the equivalence of **AddM** with a version that first obtains a borrow of the final node and then writes through it, each proved by bare structural recursion (Section 2).
- A model in Lean’s type theory (the calculus of inductive constructions with proof-irrelevant propositions and propositional extensionality), in which the backward functions reappear, giving consistency relative to Lean’s set-theoretic model, and an adequacy theorem relating the symbolic machine to concrete evaluation (Section 7). The first-order fragment of the metatheory is mechanised.
- An executable implementation of the checker in Lean 4 that checks every example in the paper (Section 8).

2 Ochr by example

This section introduces *Ochr* through its running examples. Everything is informal here; Section 3 onwards makes it precise.

2.1 Imperative code is evaluated inside types

Ochr programs are made of definitions with parameters and a body. Borrow types $\&T$ are Rust’s mutable references; places are variables x , dereferences $*p$ and the predecessor field $p.1$ of a number. A **match** on a place binds its pattern variable to a *sub-place*, not a copy, so in **match** $*x$ { $Z \Rightarrow \dots$ | $S \ p \Rightarrow \dots$ } the name p stands for the field $(*x).1$ and $\&p$ reborrows it.

Add wraps **AddM** behind a pure interface. It owns its first argument, lends it to **AddM**, and returns it:

```
Add(x : Nat, y : Nat) : Nat := AddM(&x, y); x
```

Because definitional equality is evaluation, **Id** **Nat** (**Add**(2, 3)) 5 holds by **refl**: the checker runs **Add**(2, 3), which runs **AddM** in place on a local copy, and compares the result with 5. There is nothing special about imperative code here, and nothing special about **Add** being “pure”: it is simply a program whose observable output is its result.

The machine that does this is the symbolic semantics of Aeneas’s low-level borrow calculus [10]. An environment Ω maps variables to values; borrowing x moves its content into the borrow and leaves a *loan* behind; ending the borrow moves the content back. Running **Add**(2, 3) passes through these states:

```
{ x ↦ 2 }                               // enter Add
{ x ↦ loan0 } ⊢ AddM(borrow0 2, 3)      // &x: the content moves into the borrow
{ x ↦ loan0 | x' ↦ borrow0 (S loan1), ... } // AddM matched S and reborrowed the field
...
{ x ↦ 5 }                               // borrows ended, contents returned
```

2.2 Symbolic inputs, and what a stuck call leaves behind

To reason about all numbers at once, the checker runs programs on *abstract values* σ , which play the role of Lean’s free variables. Running **Add**(σ , 0) borrows $x \mapsto \sigma$ and calls **AddM**(**borrow**₀ σ , 0), whose body immediately gets stuck: it must match on σ .

In a pure type theory a stuck term is its own normal form. Here the call has also produced something that lives in the environment, namely the final content of the place it borrowed, and that content needs a name. We *close off* the call: the call returns, and the loan it held is filled with a *sealed program*, a closed source program that owns its state and computes that content:

```
{ x ↦ loan₀ } ⊢ AddM(borrow₀ σ, 0)
→ { x ↦ [let c = σ; AddM(&c, 0); c] } ⊢ ()
```

Write $N(\sigma)$ for $[let\ c = \sigma; AddM(\&c, 0); c]$. It is a neutral value, like a stuck `Nat.rec` in Lean, and it is literally the body of `Add(σ, 0)`: the sealed program *is* the pure wrapper, generated on demand. It is also exactly the value that Aeneas’s backward function for `AddM` would compute [10], written without leaving the source language. When a later case split refines σ to $S\ \sigma'$, $N(S\ \sigma')$ runs again, now makes progress through one successor, closes off the inner recursive call, and normalises to $S\ N(\sigma')$.

2.3 Equality between computations

The type `Id Unit (AddM(x, 0)) ()` compares two computations. It is evaluated at the environment where it is formed. Each side runs on its own copy of that environment; the result and the final contents of the places the side may write form its *observation*; and `Id` computes to ordinary equality between the two observations. For a definition taking $x : \&Nat$, the checker works at the definition’s *generic call* `AddMZero(&c)` from $\{ c \mapsto \sigma \}$, so the place that may be written is `c`, the owner of x ’s loan:

```
[[AddM(x, 0)]] = ((), N(σ))      // the call is stuck, so c receives the sealed program
[()]          = ((), σ)         // nothing happens; ending x's borrow returns σ to c

Id Unit (AddM(x, 0)) () ≡ Eq (Unit × Nat) ((), N(σ)) ((), σ) ≡ Eq Nat N(σ) σ
```

The last step uses the observational computation rules for equality [4]: equality of pairs is a conjunction, reflexive equations are τ , and τ is a unit for $\wedge$. The statement “running `AddM(x, 0)` has no effect” has become the ordinary proposition that $N(\sigma)$ equals σ .

2.4 The environment performs the congruence

Now the proof:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () by x :=
  match *x { Z => refl | S p => AddMZero(&p) }
```

The match on `*x` splits on σ . In the `Z` branch the goal is `Eq Nat N(Z) Z`; $N(Z)$ runs to `Z`, the equation is reflexive and so τ , and `refl` proves it. In the `S` branch the goal is `Eq Nat N(S σ') (S σ')`, which normalises to `Eq Nat (S N(σ')) (S σ')`.

The recursive call is checked by the same rule that produced the goal: the callee’s statement is evaluated at the call site, with its parameter bound to the argument. At the call site the environment is

```
{ c ↦ loan₀ | x ↦ borrow₀ (S loan₁) } and the argument is borrow₁ σ'
```

because matching `S p` through the borrow and taking `&p` leaves the successor in place, around a loan for the predecessor. The owner of the argument’s loan is found by following it outwards: `loan₁` sits inside x ’s borrow, whose loan sits in `c`. So the recursive call’s statement observes `c`, and running `AddM` on the argument fills `loan₁` with $N(\sigma')$, after which `c` holds $S\ N(\sigma')$:

```
[[AddM(x', 0)]] at the call site = ((), S N(σ'))
[()]          at the call site = ((), S σ')
AddMZero(&p) : Eq Nat (S N(σ')) (S σ')
```

This is the goal, symbol for symbol. The successor that a pure proof would add with `cong S` was supplied by the environment. The recursive call is a proof, and proofs are erased at runtime, so the checker does not run it: its borrow argument is returned unchanged.

For comparison, here is the same theorem about the pure wrapper:

```
AddZero(x : Nat) : Id Nat (Add(x, 0)) x :=
  match x { Z => refl | S p => AddMZero(&p) }
```

`Add(x, 0)` writes nothing outside itself, so its observation is just its result, and the goal is again `Eq Nat N(σ)` `σ`, the very proposition `AddMZero`'s statement computed to; in the successor case it is `Eq Nat (S N(σ')) (S σ')`. The proof borrows the predecessor field of the *owned* number `x` and appeals to the in-place lemma. The environment again supplies the successor, so even the theorem about the pure function needs no congruence step. An induction hypothesis about a *copy* of the predecessor, `AddZero(p)`, would have type `Eq Nat N(σ') σ'`, and the successor would have to be added by hand with a congruence lemma derived from `J`. The two theorems are interchangeable: `AddZero(x) := AddMZero(&x)` type-checks, because both statements normalise to the same proposition.

2.5 Returning a borrow

A more idiomatic in-place addition first finds the final node and then writes through it:

```
TailM(x : &Nat) : &Nat by x := match *x { Z => x | S p => TailM(&p) }
AddM'(x : &Nat, y : Nat) : Unit := let t = TailM(x); *t := y
```

`TailM` returns a borrow into its argument. When it is stuck on `σ`, the place it borrowed cannot be given its final content yet, because that depends on what the caller will later write through the returned borrow. Closing off such a call returns a borrow of a fresh `k` and fills the argument's loan with a sealed program that has a *hole* `loan_k` for the returned borrow's final content:

```
{ c ↦ loan_k } ⊢ TailM(borrow_k σ)
  → { c ↦ [let c = σ; let r = TailM(&c); *r := loan_k; c] } ⊢ borrow_k [let c = σ; let r
    = TailM(&c); *r]
```

In `AddM'`, the caller writes `y` through `t` and drops it; ending `borrow_k` substitutes the final content `y` for the hole. What remains in `c` is `[let c = σ; let r = TailM(&c); *r := y; c]`: the effect of `AddM'`, as a program. The hole is the neutral form of an Aeneas region abstraction, and of a RustHorn prophecy [16]: it records only the *final* value written through the returned borrow.

The equivalence of the two additions is proved by the same bare recursion:

```
AddMEq(x : &Nat, y : Nat) : Id Unit (AddM(x, y)) (AddM'(&x, y)) by x :=
  match *x { Z => refl | S p => AddMEq(&p, y) }
```

In the successor branch, re-running `AddM'`'s sealed program on `S σ'` unfolds `TailM` once, closes off its inner call with a fresh hole, and the pending write `*r := y` then fills that hole; on the other side the recursive call's statement, evaluated at the call site, performs the same steps through the caller's borrow. Both sides arrive at `Eq Nat (S A(σ', y)) (S B(σ', y))`, where `A` and `B` are the sealed programs for the two additions on the predecessor. The theorem for an owned number follows by lending it: `AddMEqOwned(x : Nat) : Id Unit (AddM(&x, 0)) (AddM'(&x, 0)) := AddMEq(&x, 0)`.

2.6 Proofs about the current state

Dependent types are not only for stating theorems after the fact. A function can demand, as an argument, a proof about the *current* contents of a borrow. Here is in-place subtraction, which is total only when the subtrahend is no larger than the number:

```
Le(a : Nat, b : Nat) : Prop by a :=
  match a { Z => τ | S a' => match b { Z => Eq Nat Z (S Z) | S b' => Le(a', b') } }

SubM(x : &Nat, y : Nat, h : Le(y, *x)) : Unit by y :=
  match y { Z => () | S q => match *x { Z => () | S p => *x := p; SubM(x, q, h) } }
```

The type of `h` mentions `*x`, the content of the borrow at the moment of the call. Inside `SubM`, after `*x := p` has overwritten that content, the recursive call needs a proof of `Le(q, *x)` about the *new* content; the old hypothesis `h`, of type `Le(S q, S p)`, provides it, because that type was formed when `h` was bound and normalises to `Le(q, p)`.

Now a caller that first adds and then subtracts:

```
LeAdd(n : Nat, m : Nat) : Le(n, Add(n, m)) by n := match n { Z => refl | S n' => LeAdd(n', m) }

AddSub(x : &Nat, y : Nat) : Unit := let old = *x; AddM(&*x, y); SubM(x, old, LeAdd(old, y))
```

At the call to `SubM`, `*x` has just been mutated in place by `AddM`, so on symbolic input it holds the sealed program `N(σ, y) = [let c = σ; AddM(&c, y); c]`, and `SubM` demands a proof of `Le(σ, N(σ, y))`. The lemma `LeAdd` is about the *pure* `Add` of the snapshot `old`, and its type is `Le(σ, Add(σ, y))`. The two meet because `Add(σ, y)` normalises to the very same sealed program: the in-place computation and the pure one are the same program to the type checker. A proof about the pure function is accepted where a proof about the mutated state is required, with no bridging lemma. Using the snapshot after further mutation, `...; AddM(&*x, y); *x := Z; SubM(x, old, LeAdd(old, y))`, is rejected, since the requirement then mentions `Z`.

Finally, a theorem about the whole: adding `y` and then subtracting the old value leaves exactly `y`.

```
AddSubId(x : &Nat, y : Nat) : Id Unit (AddSub(x, y)) (*x := y) by x :=
  match *x { Z => refl | S p => let c = p; AddSubId(&c, y) }
```

Here the successor case copies the predecessor into a fresh place `c` instead of borrowing it in place. Borrowing would supply the surrounding `S` to the induction hypothesis, as in `AddMZero`, and the goal has no `S` to match: `AddSub` peels the successor off. Borrowing supplies the congruence and copying withholds it, and the programmer chooses.

2.7 Branching

A `match` on an abstract value in the middle of a function cannot pick an arm. Each arm is checked separately, and the rest of the function is then checked once, from the state in which the match itself has been closed off like a call. This covers borrows whose origin depends on the branch:

```
AddToOne(b : Nat, x1 : &Nat, x2 : &Nat, y : Nat) : Unit :=
  let r = match b { Z => x1 | S _ => x2 }; AddM(r, y)
```

The closed-off match returns a borrow with a hole that appears in the sealed programs for both `x1`'s and `x2`'s places; whichever the match would have chosen receives the final content. A proof about `AddToOne` splits on `b`, after which the sealed programs run and the goal becomes a statement about `AddM` alone.

3 The calculus

Oehr is a dependent type theory in the style of Lean's kernel [7], extended with places, mutable borrows and sequencing. This section gives its syntax and the runtime structures of its evaluator. We keep the core small: the only inductive types are natural numbers and the unit type, plus pairs and the propositional connectives needed by equality. Section 10 discusses what generalises directly and what does not.

3.1 Syntax

```

terms  t, u, A, B ::= x | Prop | Typei
                        |  $\Pi(x_1:A_1 \dots x_n:A_n). B$                 dependent function type
                        | fix f (x1:A1 ... xn:An) : B by xj := t    function (recursive on xj, or
not at all)
                        | t(u1, ..., un)                        saturated call
                        | Nat | Z | S t | Unit | () | A × B | (t, u) | t.1 | t.2
                        | Eq A t u | refl | J(A, a, b, P, h, t) |  $\tau$  | P ∧ Q | ⟨h, k⟩
                        | &A                                      borrow type
                        | p | &p | p := t | let x = t; u | t; u
                        | match p { Z => t | S y => u }
                        | Id A t u                                equality of computations
places p ::= x | *p | p.1

```

Figure 1: Syntax of Ochr. Eq, τ and $\wedge$ live in Prop. A fix without by is an ordinary λ .

Figure 1 gives the syntax. Types and terms share one grammar, as in any pure type system. Functions are n-ary and calls are saturated: a partial application would be a closure capturing its arguments, and a closure capturing a borrow is outside the core. A recursive function names the parameter it recurses on.

The imperative fragment is small. A *place* p is a variable, a dereference $*p$, or the predecessor field $p.1$ of a number. A place used as a term reads it; $\&p$ borrows it; $p := t$ assigns it; $\text{let } x = t; u$ introduces a new place x . The pattern variable of a *match* is a *sub-place*: in $\text{match } p \{ S y \Rightarrow u \}$, y stands for $p.1$, and nothing is copied.

$\&A$ is the type of a mutable borrow of an A . In the core, borrow types occur only as the types of variables, parameters and results, never inside another type: there are no borrows stored in data structures and no borrows of borrows. There are no shared borrows and no loops. Recursion is structural.

The propositional fragment is Lean’s: Prop is an impredicative universe with definitional proof irrelevance, erased at runtime, and Eq is its equality. Id $A \ t \ u$ compares two *computations* t and u of type A ; Section 5 shows that it is not a new primitive.

3.2 Values and environments

```

values      v, w ::= Z | S v | () | (v, w) | * | closures | types
              | borrowℓ v | loanℓ | ⊥ | n
neutrals    n    ::= σ | [t]
environments Ω   ::= a stack of frames of bindings  x : A ↦ v

```

Figure 2: Runtime structures.

The evaluator manipulates the values of Figure 2. Following the low-level borrow calculus of Aeneas [10], a borrow carries the borrowed content with it: **borrow_ℓ v** is a borrow, identified by ℓ , whose current content is v , and **loan_ℓ** is the placeholder left at the place it was taken from. When the borrow ends, its content is put back where the loan is. We depart from Aeneas in one respect: **a loan is a variable bound by its borrow**. It may occur anywhere inside a value, including inside a sealed program, and inside sealed programs it may occur more than once; ending the borrow substitutes its content for every occurrence. $\perp$ marks a place whose content has been moved out, and $*$ is the value of a proof.

Neutral values are stuck computations. An *abstract value* σ is an unknown value of a known type, playing the role of a free variable in Lean’s kernel: the checker introduces one for every parameter of a definition it checks, and nowhere else. A *sealed program* $[t]$ is a closed program whose run is stuck on an abstract value. Sealed programs are produced by closing off stuck calls (Section 4) and are the only neutral form besides abstract values. A value is *borrow-free* if it contains no borrow and no live loan.

An environment Ω is a stack of frames, one per active call, each binding variables to values. `content( $\Omega$ ,  $p$ )` follows a place through variables, borrows (`* $p$` looks inside `borrow $\ell$   $v$` at v) and successors (`$p.1$` looks inside `$S$` at v). We write $\Omega[p \mapsto v]$ for the environment with the content of p replaced.

4 Evaluation and closing off

Definitional equality in Ochr is normalisation by one deterministic machine. With effects, rewriting-anywhere is not confluent: evaluating $(\lambda y. (x := 2; y))(x := 1)$ argument-first and substituting-first leave x holding different numbers. A fixed call-by-value order restores uniqueness, and the machine then plays the role that normalisation by evaluation plays for pure type theories [1]: two terms are definitionally equal when the machine sends them to the same normal form. On concrete inputs it is an ordinary interpreter. On abstract inputs it runs until it has to inspect an abstract value, and the question the rest of this section answers is what it returns then. The judgement is big-step: $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ runs t from environment Ω to value v and environment Ω' , and a run may instead be *stuck* (it needs to inspect a neutral) or fail with a borrow error.

4.1 Borrows

$$\begin{array}{c}
\frac{\text{borrow}_{\ell} v \text{ in } \Omega}{\Omega \rightsquigarrow \Omega[\text{borrow}_{\ell} v := \perp][v/\text{loan}_{\ell}]} \text{ END} \quad \frac{v = \text{content}(\Omega, p)}{\langle \Omega, \&p \rangle \Downarrow \langle \Omega[p \mapsto \text{loan}_{\ell}], \text{borrow}_{\ell} v \rangle} \text{ BORROW} \\
\\
\frac{v = \text{content}(\Omega, p) \text{ borrow-free}}{\langle \Omega, p \rangle \Downarrow \langle \Omega, v \rangle} \text{ READ} \\
\\
\frac{\text{content}(\Omega, p) = \text{borrow}_{\ell} v}{\langle \Omega, p \rangle \Downarrow \langle \Omega[p \mapsto \perp], \text{borrow}_{\ell} v \rangle} \text{ MOVE} \quad \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \Omega_2 = \text{drop}(\Omega_1, \text{content}(\Omega_1, p))}{\langle \Omega, p := t \rangle \Downarrow \langle \Omega_2[p \mapsto v], () \rangle} \text{ ASSIGN}
\end{array}$$

Figure 3: Borrows. Every rule that accesses a place p is preceded by [Access]: end every borrow whose loan lies on the path to p or inside its content. **drop** ends the borrows inside a dropped value and fails on a live loan in a dropped owned value.

Figure 3 gives the rules for borrows. They are Aeneas’s, simplified by treating loans as variables.

Ending a borrow [End] replaces it by $\perp$ and substitutes its content for its loan. There is no side condition: if the content itself contains loans (because something has reborrowed part of it), they travel with it, and the reborrows remain valid. Ending is allowed at any time; the machine does it lazily, when a place is accessed. [Access] makes every access exclusive: before a place is read, borrowed, assigned or matched on, every borrow whose loan lies on the path to it or inside its content is ended. A borrower that has been ended holds $\perp$, and any later use of it is an error. This is the whole of the borrow checker: exclusivity of mutable borrows is not a separate analysis but a consequence of running the program.

Reading a place copies its content if the content is borrow-free [Read] and moves it out otherwise [Move]. Data is therefore copied, and borrows, which must stay unique, are moved; reborrowing is explicit (`&* $x$`). A runtime system may implement a last-use copy as a move; an affine usage discipline outside the core licenses this, so that **AddM** performs no allocation. Inside types copying is essential: `Id Nat (Add( $x$ ,  $x$ ))` x must at least be a well-formed statement, and so must $x + x = 2 \cdot x$.

4.2 Calls and matches

A call evaluates its arguments left to right, each into a temporary of the current frame, so that [Access] can see and end them: `$f(\&x, \&x)$` ends the first borrow when the second is taken, and the call then fails reading $\perp$. It then pushes a frame binding the parameters, runs the body, and pops the frame, dropping its bindings. Popping a frame that still lends out one of its owned places is an error: something would outlive the place it borrows.

Types and proofs are erased at runtime, and the machine treats them accordingly: a term whose type is a proposition, or which is a type, is evaluated on a *private copy* of the environment, argument evaluation included, and the copy is then discarded. A proof may use mutation freely on its copy, but it cannot affect the real

environment, so running a proof and skipping it are indistinguishable and the machine skips it, returning $\star$. A call whose head is itself a neutral (an abstract function, or a sealed program) is stuck at once.

`match p { Z => t | S y => u }` inspects the head of `p`'s content. `Z` selects the first arm, and `S v` selects the second with `y` standing for the sub-place `p.1`. Through a borrow, `match *x` leaves the successor in the borrowed content, so `&y` then reborrows the predecessor field in place: $x \mapsto \text{borrow}_0 (S \text{ loan}_1)$ and the new borrow is `borrow1 v`. If the head is a neutral, the run is stuck.

4.3 Closing off

When the body of a call is stuck, the call itself is stuck, and so is everything waiting for it. A pure type theory would return the stuck call as a neutral term. An effectful call cannot simply be returned: part of what it produces is the final content of the places it borrowed, and the caller's environment needs those contents before it can continue. We *close off* the call instead.

the body of $f(w_1, \dots, w_n)$ is stuck, $w_i = \text{borrow}_{\ell_i} u_i$ for $i \in I$, u_i loan-free

$$L := \text{let } c_i = u_i \quad (i \in I) \quad C := f(a_1, \dots, a_n), \quad a_i = \begin{cases} \&c_i & \text{if } i \in I \\ w_i & \text{otherwise} \end{cases}$$

result type	result	each loan_{ℓ_i} becomes
Unit	<code>()</code>	$[L; C; c_i]$
borrow-free D	$[L; C]$	$[L; C; c_i]$
$\&T$	$\text{borrow}_k [L; \text{let } r = C; *r]$	$[L; \text{let } r = C; *r := \text{loan}_k; c_i]$

Figure 4: [Close]. The partial run of the body is discarded; the borrows `borrowℓi` are consumed; `k` is fresh.

Figure 4 gives the rule. Take the call at the point where its arguments have been evaluated. For each borrow argument `borrowℓi ui`, the canonical program `L; C` recreates the situation the call was in, but with the borrowed contents owned by fresh local places `ci`: it moves each `ui` into `ci` and calls `f` on `&ci`. This program is closed. Running it and reading `ci` afterwards computes exactly what the call would have left behind the borrow, so each loan is filled with the sealed program $[L; C; c_i]$. The call returns the sealed program $[L; C]$, or `()` if its result type is **Unit**.

Sealed programs are the backward functions of Aeneas, written in the source language. For `AddM(borrow0 σ, y)`, the loan receives $[\text{let } c = \sigma; \text{AddM}(\&c, y); c]$, which is the body of `Add(σ, y)`. Where Aeneas's translation would emit `add_m_back σ y`, Ochr writes the program that defines it. Nothing about the call is lost: when `σ` is later refined, the sealed program runs again.

Returning a borrow. If `f` returns a borrow, the contents of the places it borrowed are not determined when the call returns: they depend on what the caller will write through the returned borrow before it ends. The rule therefore returns a borrow of a fresh `k`, whose content is the sealed program for the returned borrow's initial content, and fills each `loanℓi` with a sealed program that has a *hole*: it writes `loank` through the returned borrow before reading `ci`. Because loans are variables, the hole is filled by the ordinary [End] rule: when the caller drops the returned borrow with final content `w`, `w` is substituted for `loank`. This records only the *final* value written through the returned borrow, which is all that the backward function of Aeneas takes as input, and the hole is the neutral form of an Aeneas region abstraction and of a RustHorn prophecy [16]. If `f` has several borrow arguments, the hole occurs in each of their sealed programs, and one [End] fills them all.

The precondition that each `ui` is loan-free is guaranteed by [Access], which ended those loans when the argument was read or borrowed. Without it, a live loan would be copied into a sealed program and survive the borrow that binds it.

4.4 Sealed programs and refinement

Values are kept in normal form. The normal form of a sealed program $[L; C; K]$ is obtained by running it from the empty environment, with one restriction: its *head call* `C` may not itself be closed off (calls made inside `C`'s body may). This is the analogue of the guard on fixpoint unfolding in the calculus of inductive constructions.

Without it, a sealed program would re-close into itself forever; with it, a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run (a hole whose returned borrow is still live) is *inert*: the run treats it like an abstract value.

A *refinement* substitutes a constructor pattern for an abstract value, $\sigma := Z$ or $\sigma := S \sigma'$, when a case split learns it (Section 6). Substitution into a sealed program is followed by normalisation, which runs the sealed program again. For $N(\sigma) = [\text{let } c = \sigma; \text{AddM}(\&c, \theta); c]$ and $\sigma := S \sigma'$, the head call **AddM** now unfolds, matches the successor, reborrows the predecessor, and makes the inner recursive call, which is stuck on σ' and is closed off; so $N(S \sigma') = S N(\sigma')$. Normal forms of this shape are what makes the induction of Section 2 go through. The key property is that refinement commutes with closing off: refining the arguments of a closed-off call and re-normalising gives the same normal form as closing off the call on the refined arguments (Section 7).

Stuck blocks. A stuck **match** that is not the body of a call is closed off in the same way, as the body of a call to an anonymous function of its free places. The function takes its free places the way Rust infers closure captures: a place the match moves out of, in any arm, is moved in; a place it writes or borrows is passed by borrow; a place it only reads is copied. This covers stuck matches inside types and, during checking, a stuck match that is followed by more code (Section 6).

5 Observational equality

Id $A \ t \ u$ states that the computations t and u , both of result type A , are indistinguishable from where the statement is made: run from the current environment, they return equal values and leave equal contents in every place either of them may write. This section defines that notion precisely and shows that it reduces to ordinary propositional equality.

5.1 Footprints and observations

A computation affects the world only through the places it names. The places a term may **write** are read off its syntax: they are the free places that appear under $\&_$ or on the left of $:=$, and the borrow variables it mentions, since a borrow grants write access to someone else's place. Each of these is traced back to the owned place that will receive the write. For a place rooted at an owned variable x , that is x . For a borrow variable holding $\text{borrow}_\ell v$, it is the set of *owners* of ℓ :

$$\text{owners}(\ell) = \bigcup_{\text{occurrences of } \text{loan}_\ell} \begin{cases} \text{owners}(m) & \text{if the occurrence lies inside the content of } \text{borrow}_m \\ \{x\} & \text{if it lies inside the owned binding } x \end{cases}$$

Following loans outward through enclosing borrows is what connects a reborrow with the place it ultimately writes. In the successor case of **AddMZero**, the argument $\text{borrow}_1 \sigma'$ has its loan inside the content of x 's borrow_0 , whose loan is in the generic caller's c ; so the owner of the recursive call's argument is c , the same place the goal observes. The owners form a set because a hole left by a returned borrow can occur in several sealed programs at once. Observing only one of them would be unsound: a call that returns a borrow into one of two arguments could then be proved to leave the other unchanged whichever it picked.

The *footprint* $W(t, u)$ is the union of the owners of the places t or u may write. It is syntactic, so it does not change when a later case split refines an abstract value, and it does not depend on unrelated places in the environment.

The *observation* of t at Ω on footprint W runs t on a private copy of Ω , ends every borrow that remains, and reads off the result and the contents of the footprint:

$$\llbracket t \rrbracket_\Omega^W = (v, \Omega'(k)_{k \in W}) \quad \text{where } \langle \Omega, t \rangle \Downarrow \langle \Omega'', v \rangle \text{ and } \Omega'' \overset{*}{\rightsquigarrow} \Omega' \text{ with no borrows left}$$

If t is itself stuck outside any call, it is first closed off as a stuck block, so an observation always exists. Ending every remaining borrow makes the observation independent of which borrows happened to be live when t finished; that the order in which they are ended does not matter is a theorem (Section 7).

5.2 Id is equality of observations

$$\text{Id } A \ t \ u \equiv \text{Eq } (A \times T_W) \llbracket t \rrbracket_{\Omega}^W \llbracket u \rrbracket_{\Omega}^W \quad W = W(t, u), \text{ both sides run from } \Omega \text{ on independent copies}$$

$$\text{Eq } (A \times B)(a, b)(a', b') \equiv \text{Eq } A \ a \ a' \wedge \text{Eq } B \ b \ b' \quad \text{Eq } A \ a \ b \equiv \top \quad \text{if } a \equiv b \quad \top \wedge P \equiv P \equiv P \wedge \top$$

Figure 5: **Id** computes to **Eq** between observations ($\top_W$ is the product of the types of the places in W ; for empty W the pair is just A), and **Eq** computes on pairs and on reflexive instances. **refl** : $\top$.

Figure 5 is the whole of **Id**. Both sides run from the same environment, each on its own copy; sharing one copy and running the sides in sequence would let the first side’s effects change what the second observes. The result type A must be borrow-free: a borrow has no meaning once the computation that produced it has finished.

The equality **Eq** on the right is Lean’s, living in the proof-irrelevant universe **Prop**. We add three conversion rules in the style of observational type theory [4, 19]: equality at a product is a conjunction, a reflexive equation is $\top$, and $\top$ is a unit for $\wedge$. Their only role is to strip away the parts of an observation that no side changed, so that **Id Unit (AddM(x, 0)) ()** computes to the single interesting equation

$$\text{Eq } (\text{Unit} \times \text{Nat})((), N(\sigma))(\(), \sigma) \equiv \top \wedge \text{Eq Nat } N(\sigma) \ \sigma \equiv \text{Eq Nat } N(\sigma) \ \sigma.$$

Each rule identifies two propositions with the same truth value, so all three hold in the proof-irrelevant set model of Lean’s type theory [7]. Accordingly **refl** proves $\top$, and by conversion every reflexive equation. Transport $J(A, a, b, P, h, t)$ takes its endpoints explicitly, since a reflexive $\text{Eq } A \ a \ a$ computes to $\top$ and no longer records them.

Three consequences are worth drawing out.

Effects are outputs. The observation turns every write to a place in the footprint into an extra component of the result. This is the sense in which **Ochr** has no pure/impure divide: a computation’s effects are part of its value as seen by **Id**, and the programmer never declares them.

Owned locals are observed. The footprint includes places the current function owns, not only the places behind its borrow parameters. This is forced. A context such as $(\Box; x)$, which reads x afterwards, distinguishes $x := 6$ from $()$; if **Id** ignored owned locals it would equate them, and transport along that equation would prove $\text{Eq Nat } 6 \ 5$.

Unobserved moves are harmless. Reading a borrow variable moves it, so the two sides of an **Id** may consume different borrow variables. Borrow variables are never in the footprint (their owners are), and ending the remaining borrows puts every borrowed content back in its owner, so what one side consumed and the other did not makes no difference to the comparison.

6 Typing

Typing is the machine run on abstract inputs, with case splitting where the checked program itself inspects an abstract value. The judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ says that from Ω the term t runs to the value v of type A , leaving Ω' . It returns the value, not just the type, because later types may depend on it: after **let** $z = (\text{let } y = \&x; *y := 2; x)$, the type $\text{Id Nat } z \ 2$ holds by **refl** only because the checker knows that z is 2. The rules for the basic forms are the evident ones, each the corresponding machine rule annotated with types, and we omit them. Figure 6 gives the rules that are particular to **Ochr**.

[**Call-type**] At the point where the arguments of $f(\bar{a})$ have been evaluated, with $f : \Pi(\bar{x} : \bar{A}) . B$: push a frame binding each x_i to a_i 's value, evaluate B there on a private copy of the environment, and pop. The result is the type of the call. Each a_i must have type A_i , evaluated with the earlier parameters bound.

[**Def**] $\text{fix } f (\bar{x} : \bar{A}) : B \text{ by } x_j := b$ is checked at its *generic call*: from an environment with a fresh owned place $c_i \mapsto \sigma_i$ for each borrow parameter $x_i : \&T_i$, the call $f(\bar{a})$ with $a_i = \&c_i$ for borrow parameters and $a_i = \sigma_i$ otherwise. The goal is the [Call-type] of this call. The body runs in the pushed frame, which is then dropped; its result's type must be convertible to the goal, as refined by the splits on the way.

[**Split**] When the checked program matches on a place whose content has an abstract head σ , each arm is checked with the refinement $\sigma := Z$, respectively $\sigma := S \sigma'$ for fresh σ' , applied to the environment, the goal and every stored type. If the head is a sealed program, every occurrence of it is first replaced by a fresh σ . If the match is followed by more code, that code is checked once, from the state in which the match has been closed off as a stuck block.

[**Rec**] In the body of $\text{fix } f \dots \text{by } x_j$, f occurs only as the head of a call, and each recursive call passes in position j a value, or a borrow of a value, that is a strict subterm of x_j 's entry value σ_j as refined so far.

Figure 6: The typing rules particular to Ochr.

6.1 One rule for goals and induction hypotheses

A definition is checked by running it at its most general call [Def]. For $\text{AddMZero}(x : \&\text{Nat})$ that is $\text{AddMZero}(\&c)$ from $\{ c \mapsto \sigma \}$: the parameter becomes a borrow of a fresh owned place, which stands for whatever the real caller lends. The goal is then computed by [Call-type], the rule that types every call.

The same rule types the recursive call in the successor case, at a different environment. There the argument $\&p$ borrows the predecessor field of c 's content, so evaluating the callee's statement at the call site observes c through the successor that surrounds the field. This is how the induction hypothesis arrives already wrapped in S . The generic call and the recursive call are two instances of one rule, which is why the goal and the hypothesis meet without a congruence step. That the callee's statement, proved at its generic call, remains true at every call site is the frame property of Section 7: a call affects only what it is passed, so the caller's observation is the callee's, placed in a context.

6.2 Types are formed once

A type is evaluated once, when it is formed, on a private copy of the environment, and becomes a closed statement about values. Later mutation cannot change it. This includes the codomain of a Π -type, which is a closure over the values of the variables it mentions; only its parameters are bound later, at each call. The goal of a definition is formed at its generic call, before the body runs, so a body that mutates its argument and then proves something about the new contents does not prove the goal: $\lambda(x : \&\text{Nat}). (*x := 5; \text{refl})$ does not have type $\Pi(x : \&\text{Nat}). \text{Id Nat } (*x) 5$.

6.3 Case splitting and joins

A *match* in the checked program on an abstract value cannot pick an arm, so each arm is checked with the abstract value refined [Split]. What follows the match is checked once, from a state in which the match itself has been closed off as a stuck block. The closed-off match is precise: its sealed programs contain the match, and they reduce as soon as a later split decides it. There is no join of environments and no loss of information, and a borrow whose origin depends on the branch is handled like any other returned borrow. AddToOne of Section 2 is accepted, and so is any function whose body is split into a helper at a match: a match and a call to a function containing it are treated alike. Errors in the code after a match are reported once, at the line where they occur, not once per arm.

6.4 Erased terms leave no trace

Types and proofs are erased at runtime. The machine mirrors this exactly: it evaluates a type, or a term whose type is a proposition, on a private copy of the environment, argument evaluation included, and discards the copy. Two things follow. A formed type is a closed statement about values, as described above. And a proof can have

no effect on the program around it: it may mutate places freely, but only on its own copy. Running a proof and skipping it are therefore indistinguishable, and the machine skips proofs, which is what the compiled program does too.

This is one principle, not a restriction on proofs. **AddMZero**’s successor case borrows the field **p** to pass it to the induction hypothesis; the borrow happens on the private copy, and the real environment is untouched. Nothing in a program is marked pure, and any program may appear in a statement.

6.5 Why each condition is there

Each side condition above was added in response to a concrete false proof or run-time error found while designing the calculus. We list them because together they delimit the design, and because each is a regression test for the implementation (Section 8).

- **Π -types capture values.** If the codomain of a hypothesis $h : \Pi(_ : \text{Unit}). \text{Id Nat } x \text{ Z}$ were re-evaluated at each call, then $x := S \ x$; $h()$ would produce a proof of $\text{Id Nat } (S \ Z) \ Z$ from h , which was proved when x was Z .
- **Recursion on entry values.** A syntactic structural check accepts $f(x : \&\text{Nat}) := *x := S \ *x$; $\text{match } *x \{ S \ p \Rightarrow f(\&p) \}$, whose recursive argument is the parameter’s original value, and with it a proof of $\Pi(n : \text{Nat}). \perp$. The recursive function must also never escape as a value, or the check can be bypassed through a higher-order call.
- **Erased terms run on a private copy.** Proof irrelevance at a Π -type over a borrow identifies $\lambda x. *$ with $\lambda x. (*x := 7; *)$; if calls to these could affect the caller, transport along that identification proves $\perp$. Stipulating only that proof *calls* are skipped is not enough: a proof block that writes, closed off after a split and then refined, would be skipped, but run directly it would not, and a closed proof of $\text{Id Nat } (S \ Z) \ Z$ results. Running every erased term on a private copy makes skipping a consequence rather than a stipulation, and makes it commute with refinement. For the same reason a type must be formed on a private copy: a type that writes would otherwise make the checker and the compiled program disagree.
- **All owners are observed.** A function returning a borrow into one of two arguments leaves its hole in both. A footprint that observed only one owner lets [Call-type] prove $\text{Id Nat } (S \ Z) \ Z$.
- **Access is exclusive and closing off needs loan-free arguments.** Moving a borrow whose content still holds a live loan into a call, and closing that call off, copies the loan into a sealed program; after refinement the sealed program disagrees with running the call, and the compiled program writes through an ended borrow.

7 Metatheory

We show that *Ochr* is consistent and that its symbolic evaluator computes what programs actually do, using a model in Lean’s type theory. The model reads a program that mutates what it borrows as a pure function from the current contents of its borrows to its result and their final contents. This is Aeneas’s functional translation [10]; its backward functions appear here and only here. The results below say that sealed programs *are* backward functions (Theorem 2). They say that a call affects only what it is passed (Theorem 3), that borrows may end in any order (Theorem 6), and that symbolic evaluation commutes with instantiation on everything a type can observe (Theorem 7). And they say that typing and conversion are preserved, so no false proposition has a closed proof (Theorem 9, Corollary 10).

7.1 The model

Target. We translate into CIC_L , the type theory of Lean 4 [17] as axiomatised by Carneiro [7]. It has universes $\text{Prop} : \text{Type}_0 : \text{Type}_1 \dots$, an impredicative **Prop** with definitional proof irrelevance, inductive types, and the axiom **propext**, which says that logically equivalent propositions are equal. Computations need nothing extensional: each machine step becomes an intensional conversion (Theorem 2). Propositions, however, need **propext**, because *Ochr*’s conversion identifies propositions that CIC only proves equivalent: the rules for **Eq** (Section 5) and, above all, a codomain computed at a call site with the codomain proved at the generic call (Theorem 3). Conversion is therefore preserved into ECIC_L , which adds equality reflection, or into CIC_L up to

transports along **propext** equalities, as in the elimination of reflection [22]. Both have Carneiro’s set model. In it propositions with the same truth value are equal sets, and an inhabited equation forces its two sides equal.

Types. Data and propositions translate homomorphically, $\mathbf{Nat}^\dagger = \mathbb{N}$, $(\mathbf{Eq} \ A \ a \ b)^\dagger = (a^\dagger = b^\dagger)$, and so on. Consider a function type with borrow parameters $x_i : \&\mathbf{T}_i$ at the positions I . Let $D_i = T_i^\dagger$ for $i \in I$ and $D_j = A_j^\dagger$ otherwise, and let $\mathbf{Fin} = \prod_{i \in I} T_i^\dagger$, the final contents of the borrowed places. It translates to forward and backward functions:

$$(\Pi(\bar{x} : \bar{A}).B)^\dagger = \Pi(\bar{d} : \bar{D}).\text{Out}_B(\bar{d}), \quad \text{Out}_B(\bar{d}) = \begin{cases} B^\dagger(\bar{d}) & \text{if } B \text{ is a proposition,} \\ B^\dagger(\bar{d}) \times \mathbf{Fin} & \text{if } B \text{ is borrow-free data,} \\ T^\dagger \times (T^\dagger \rightarrow \mathbf{Fin}) & \text{if } B = \&T, \text{ with } T^\dagger \rightarrow \mathbf{Fin} \text{ injective.} \end{cases}$$

Here $B^\dagger(\bar{d})$ translates B ’s normal form at the generic call of [Def], with the borrowed places holding $\bar{d}$. We write fwd_f and back_f^i for the result and the i -th final content of $f^\dagger$; in the third case back_f^i is a function of the returned borrow’s final value. Three features carry weight.

- *A proof is not a state transformer.* Erased terms run on a private copy (Section 6), so a proposition-valued function has no $\mathbf{Fin}$ component, and proof irrelevance at its type is sound. If proofs had effects, proof irrelevance would identify $\lambda x. * \text{ with } \lambda x. (*x := 7; *)$ at $\Pi(x : \&\mathbf{Nat}). \top$, although they are different functions into $\mathbb{N}$, and transport would prove $\perp$.
- *Backward functions await the final value* of the returned borrow, which the caller supplies later. Aeneas’s region abstraction is here just a λ .
- *Injectivity* is where the model is a logical relation rather than a translation, and it is forced. By [Call-type], Ochr refutes $\mathbf{Id} \ \mathbf{Unit} \ (\text{let } r = h(\&a); *r := Z) \ (\text{let } r = h(\&a); *r := S \ Z)$ for every *abstract* $h : \Pi(x : \&\mathbf{Nat}). \&\mathbf{Nat}$, which says that h ’s backward function is injective. That fails for arbitrary CIC functions but holds for definable ones (Lemma 5), much as thunkability [20] and parametricity [6] hold only on the definable part of a function space.

Environments and terms. Abstract values become CIC variables, listed in Γ_Ω . A loan name ℓ becomes a variable h_ℓ , a *hole* for the final content of borrow ℓ , wherever the loan sits. The *view* $\Omega^\dagger$ is the tuple of translated contents, with $(\text{borrow}_\ell v)^\dagger = v^\dagger$ (a borrow is seen as its current content) and $\text{loan}_\ell^\dagger = h_\ell$. The *resolution* ρ_Ω , the substitution $h_\ell := v_\ell^\dagger[\rho_\Omega]$ for each $\text{borrow}_\ell v_\ell$ in Ω , means “end every borrow”. A run fixes the *shape* of its environments (where bindings, borrows and loans sit) independently of the abstract values; that is what the borrow checker guarantees. On fixed shapes a term becomes a state-passing function on views (Figure 7). A sealed program means what it computes, $[t]^\dagger = \text{Run}(t)$, the result of $t^\dagger$ run from the empty view. A definition becomes strong recursion on its decreasing parameter’s entry value. An observation becomes the tuple of its result and resolved footprint, and $\mathbf{Id}$ becomes the CIC equation between two such tuples.

$$\begin{aligned} \text{End}_\ell : \quad s &\mapsto (s \setminus \text{holder}(\ell))[h_\ell := s.\text{holder}(\ell)] \\ \text{match } p\{\dots\} : \quad s &\mapsto \text{natCase } (s.p) \left(t_Z^\dagger(s[p := 0]) \right) \left(\lambda \sigma'. t_S^\dagger(s[p := \text{succ } \sigma']) \right) \\ f(\bar{a}), B \text{ data} : \quad s &\mapsto \text{let } (r, \bar{\varphi}) = f^\dagger(\bar{u}, \bar{w}) \text{ in } (r, (s \setminus \bar{a})[h_{\ell_i} := \varphi_i]) \\ f(\bar{a}), B = \&T : \quad s &\mapsto \text{let } (c, \beta) = f^\dagger(\bar{u}, \bar{w}) \text{ in } (c, (s \setminus \bar{a})[h_{\ell_i} := \beta_i(h_k)]) \\ t \text{ a type or a proof} : \quad s &\mapsto (v^\dagger, s) \end{aligned}$$

Figure 7: Key state-passing clauses; $\bar{u}$ are the current contents of the borrow arguments, with loans ℓ_i , $\bar{w}$ the other arguments, k fresh. Ending a borrow is a substitution; a call is one CIC application, Aeneas’s forward and backward translations fused, with the region abstraction as the substitution $h_{\ell_i} := \beta_i(h_k)$; an erased term leaves the view unchanged.

Well-formedness. An environment is *well formed* when four conditions hold. Each borrow is held once. Every loan is bound by a borrow it holds, or lies inside a sealed program, and only there may a loan occur twice. The relation “loan ℓ occurs in the content of borrow m ” is acyclic. And every value read, moved, borrowed or passed is loan-free, as [Access] enforces.

Lemma 1 (invariance and termination). Machine steps preserve well-formedness. For programs accepted by [Rec], every run terminates in a value, a borrow error, or a stuck state.

Proof sketch. Acyclicity is preserved because a loan enters a borrow's content only through a reborrow of a sub-place, and [Access] stops reads from copying loans. Termination is a reducibility argument over the simple structure of runtime values. It reduces to well-founded recursion, which is exactly [Rec]: every recursive call is strictly below the entry value, and f occurs nowhere else. ■

7.2 Sealed programs are backward functions

Theorem 2 (simulation). If Ω is well formed and $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ by the symbolic machine, then $t^\dagger(\Omega^\dagger) \equiv (v^\dagger, \Omega'^\dagger)$ in CIC_L , in context Γ_Ω and the holes of Ω . In particular, the sealed programs that [Close] produces at a call $f(\bar{a})$ with borrow contents $\bar{u}$ and other arguments $\bar{w}$ satisfy

$$[\text{L}; \text{C}]^\dagger \equiv \text{fwd}_f(\bar{u}, \bar{w}), \quad [\text{L}; \text{C}; c_i]^\dagger \equiv \text{back}_f^i(\bar{u}, \bar{w}), \quad [\text{L}; \text{let } r = \text{C}; *r := \text{loan}_k; c_i]^\dagger \equiv \text{back}_f^i(\bar{u}, \bar{w})(h_k).$$

The same holds for the concrete machine.

These equations are definitional. A sealed program is not an approximation of a backward function; it is a name for one, written in the source language.

Proof sketch. By induction on the derivation. Reads, borrows and assignments project and update the view, [End] is the substitution of Figure 7, a match on a constructor is ι -reduction, and an unfolding call is $\delta\beta$ followed by the induction hypothesis. For [Close], the equations hold by unfolding Run. The call clause yields these projections whether or not $f^\dagger(\bar{u}, \bar{w})$ reduces further, which is why discarding the partial run is harmless. [Seal] runs t , and the induction hypothesis on that run gives the normal form. Translation commutes with refinement because abstract values and loans are variables.

The hardest case ends a borrow k whose loan is a hole inside sealed programs. The model substitutes $h_k := w^\dagger$ into $\text{Run}(t)$; the machine substitutes w into t and runs it again. The two agree, $\text{Run}(t)[h_k := w^\dagger] \equiv \text{Run}(t[w/\text{loan}_k])$, only because the hole is inert *during* the run of t : [Seal] treats a loan whose borrow is outside the run like an abstract value. A run able to end k would reach outside the sealed program, and $\text{Run}(t)$ would not be a function of t . ■

7.3 A call affects only what it is passed

Theorem 3 (frame). Let Ω be well formed.

1. *Locality.* Suppose $\Omega = \Omega_1 \uplus \Omega_2$, t 's free variables are bound in Ω_1 , and every loan occurring in Ω_1 has its borrow in Ω_1 . Then $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ iff $\langle \Omega_1, t \rangle \Downarrow \langle \Omega'_1, v \rangle$ with $\Omega' = \Omega'_1 \uplus \Omega_2\theta$. Here θ fills the loans in Ω_2 of the borrows the run ended.
2. *Call effect.* A call runs as its body from its own frame alone, then substitutes each borrow argument's final content for its loan: the call clause of Figure 7.
3. *Call typing.* For $f : \Pi(\bar{x} : \bar{A}).B$ called at Ω , once its arguments are evaluated, every Id -atom of B observed at Ω equals the atom observed at the generic call, mapped through the context Ctx . Ctx resolves *all* owners of the arguments' loans around their final contents. So, if Ctx is injective, $B@ \Omega^\dagger = B^\dagger(\bar{u})$ by **proptext**.

Lemma 4 (owners and injectivity). Ctx is jointly injective when it resolves every owner of every hole it carries, and in general only then.

Lemma 5 (backward functions are injective). For every definable $f : \Pi(\bar{x} : \bar{A}).\&T$ and all $\bar{d}$, the backward map of $f^\dagger(\bar{d})$ is jointly injective.

Part (3) is what [Call-type] relies on. It is why an induction hypothesis arrives wrapped in its caller's context, which in **AddMZero** is $((), S \square)$.

Proof sketch. (1) By induction on the run: with no globals and no closure capturing a borrow, a rule touches only places reachable from t . (2) is (1) for the callee's frame, which is loan-closed because arguments are loan-free. (3) Both observations run on private copies and, by (1), change nothing outside the arguments. With Lemma 4 the two Eq -atoms are equivalent, and congruence does the rest. For Lemma 4, Ctx composes constructors, backward functions at their hole (Lemma 5, or the model's restriction), and components constant in the hole. Those keep

joint injectivity only if the owner holding the hole is kept, hence owner sets (Section 5). With one owner of a two-argument returned borrow observed, [Call-type] proves Id Nat (S Z) Z . Lemma 5 is the fundamental lemma of the relation, by induction on typing. A returned borrow's hole sits under constructors or at an injective backward function, and no rule discards its final value ([Access] ends inner loans before overwriting or dropping). ■

The hardest case is (2) when f returns a borrow. As the frame pops, the parameter's borrow still holds the returned borrow's loan. [End] moves that content, loan included, into the caller's owner, which ends up holding $K[\text{loan}_r]$, a value that depends on something the caller has not yet produced. In the model this is $\beta_i(h_r) = K[h_r]$. Its injectivity ranges over every future of r , so it takes an induction over definitions (Lemma 5) rather than an inspection of Ω .

7.4 Borrows may end in any order

Theorem 6 (canonical observation). Let Ω be well formed.

1. Ending two borrows in either order and re-normalising gives environments with the same resolution.
2. If a run of t from Ω succeeds while ending some borrows earlier than the lazy strategy would, then the lazy run succeeds too, with the same borrow-free result and the same resolution.

Hence the observation $\llbracket t \rrbracket_{\Omega}^W$ does not depend on when borrows end, and **Id** is well defined.

Proof sketch. (1) Up to re-normalisation, ending ℓ is the substitution $h_{\ell} := v_{\ell}^{\dagger}$, and two such substitutions commute by acyclicity. (2) Suppose the first run ends ℓ at a point A , while the lazy run ends it later, at a demand, at a drop, or at the final resolution. After A the first run cannot use ℓ , since it would read $\perp$. Nor can it reach the positions of ℓ 's loan: any access to their container would make the lazy run end ℓ there too. So every step of the first run is available to the lazy run, and both substitute the same content for ℓ 's loan. ■

For (1), the hardest case is a hole inside a sealed program. Ending its borrow re-runs the sealed program, and the order of two such re-runs must not matter. It does not: a completed sealed run ends its own borrows before reading its result, so its normal form is a resolved value, and **Run** is compositional (Theorem 2). Either order therefore computes **Run** of the fully substituted program. Part (2) is the part the rest of the metatheory leans on: it absorbs the extra borrow endings of the symbolic machine in Theorem 7.

7.5 Symbolic evaluation commutes with instantiation

A *refinement* α of Ω substitutes constructor patterns or closed values for abstract values, closed definable functions for abstract functions, and values for holes in sealed programs. $\Omega \downarrow$ is Ω with its sealed programs re-normalised, and $\approx$ is equality up to renaming of loans.

Theorem 7 (naturality). Let Ω be well formed, $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ by the symbolic machine, and α a refinement of Ω .

1. *Operationally*, $((\Omega' \alpha) \downarrow, (v \alpha) \downarrow)$ is, up to $\approx$, the result of a run of $t \alpha$ from $(\Omega \alpha) \downarrow$ that ends some borrows earlier than the lazy strategy. So the lazy run $\langle (\Omega \alpha) \downarrow, t \alpha \rangle \Downarrow \langle \Omega'', v'' \rangle$ exists. It has the same resolution, and when v is borrow-free it has the same result.
2. *Observationally*, for every footprint W , re-normalising the instantiated observation gives the observation of the instantiated term: $(\llbracket t \rrbracket_{\Omega}^W \alpha) \downarrow = \llbracket t \alpha \rrbracket_{\Omega \alpha}^W$.

Corollary 8 (adequacy). If α is ground, the lazy run of $t \alpha$ closes nothing off: it is a concrete run, it terminates, and it computes the instantiated symbolic observation. Every observation, and hence every **Id**, that the checker computes on abstract inputs is therefore the observation the program makes on every concrete input. The checker is sound in this sense, but it is not complete (see the remark below).

Proof sketch. Part (1) is by induction on the symbolic derivation. Every step except closing off commutes with α on the nose. α substitutes atoms, and [Match], the only rule that is not uniform in atoms, takes an arm only on a constructor, which α preserves. At a [Close] of a call C with argument contents $\bar{u}$, the refined run reaches C with $\bar{u} \alpha$ and runs f 's body alone (Theorem 3). Re-normalising the sealed programs runs the same body on the same values. [Seal] unfolds the head call exactly once, and closes off inner calls exactly where the direct run does. By determinism, both stop at the same stuck match or complete with the same results. They can differ in one place only: where a returned borrow's loan lives. The concrete run puts it into the one owner the borrow

actually points into. The symbolic hole sits in the fill of every borrow argument the borrow *might* point into, and an [Access] to any of those places ends the borrow. So the symbolic run can end a returned borrow at an access that the concrete run performs without ending it. That is an extra [End] step, and nothing else changes. Part (2) follows from (1) and Theorem 6 (2), since an observation resolves every borrow. ■

The hardest case is again a returned borrow, and it has two sub-cases.

No non-target owner accessed. Suppose no possible owner other than the target was accessed before α . Then re-normalising reproduces the concrete state up to renaming. The concrete run pops f 's frame while its parameter's borrow holds the returned loan loan_q , so the target owner receives $K[\text{loan}_q]$. On the symbolic side, re-normalising $[L; \text{let } r = C; *r := \text{loan}_k; c_i]$ runs C , leaving c_i holding $K[\text{loan}_{q'}]$. It then writes the inert loan_k through r and reads c_i , where [Access] ends the run's own q' . The result is $K[\text{loan}_k]$, which matches $K[\text{loan}_q]$ under $k \leftrightarrow q$. This rests on loans being variables, on [Access] ending only the run's own loans, and on [Seal] not closing off its head call.

A non-target owner accessed. Suppose the symbolic run accessed a possible owner that α reveals is not the target. Then it has already ended the borrow, and only the resolutions of the two states agree.

Remark (holes over-approximate the target of a returned borrow). This example is machine-checked in the Lean development.

The setup:

- $\text{Pick}(n, x, y) := \text{match } n \{ Z \Rightarrow x \mid S _ \Rightarrow y \};$
- $\Omega = \{n \mapsto \sigma, a \mapsto 1, b \mapsto 2, r \mapsto (), z \mapsto ()\};$
- $t = r := \text{Pick}(n, \&a, \&b); z := b;$
- $\alpha = (\sigma := Z).$

The symbolic run:

- Pick is stuck on σ and is closed off.
- The hole of its returned borrow sits in the fills of both a and b , since either may be the target.
- Reading b therefore ends the returned borrow, and $r \mapsto \perp$.
- After refinement, the state is $\{a \mapsto 1, b \mapsto 2, r \mapsto \perp, z \mapsto 2\}.$

The concrete run of $t\alpha$:

- Pick returns a borrow of a , so b holds no loan and r stays live.
- The final state is $\{a \mapsto \text{loan}_q, b \mapsto 2, r \mapsto \text{borrow}_q 1, z \mapsto 2\}.$

No renaming of loans relates these two states, though they resolve to the same contents. That is why Theorem 7 is stated up to resolution.

The cost is completeness, not soundness. The program $r := \text{Pick}(n, \&a, \&b); z := b; \text{match } n \{ Z \Rightarrow *r := 5 \mid S _ \Rightarrow () \}$ runs for every concrete n . But the checker rejects it: r is already $\perp$ when the Z arm uses it. The error lands on that use. It goes away if the program uses r before reading b , or splits on n before the call. Recovering the program as written would need holes that record under which refinement they belong to which owner. Rust's borrow checker draws the same line: after $\text{let } r = \text{if } n == 0 \{ \&\text{mut } a \} \text{ else } \{ \&\text{mut } b \}$, it treats r as borrowing from both a and b for as long as r is live, and rejects reading b in between. The checker is as conservative as Rust here, and the concrete machine is more permissive than both.

Thunkability is naturality. In Pédrot and Tabareau's forcing model of dependent call-by-push-value, a computation is a family over forcing conditions. It is *thunkable* (it behaves like a value) exactly when it is *natural*: running then restricting equals restricting then running [20:Prop. 18]. Ochr's type-level evaluation has this shape. The normaliser runs at a condition Ω , which records which abstract values exist and what is known of them, and refinements are the restriction maps. [Split] refines stored types in lockstep with the term, and so plays the role of their dependent **let**.

A type contains only observations, and observations are resolved. So part (2) of Theorem 7, the naturality square for everything a type can see, commutes on the nose. That is the precise sense in which types are free of effects although they run effectful programs. Operational states commute only up to extra borrow endings, by part (1). The difference is invisible to types and costs only the completeness just described.

This is a correspondence of invariants, not an embedding into their calculus, but it is a usable design test: a rule may act in type-level evaluation only if it is natural on observations. Three candidate rules failed it:

- joining a stuck match’s results into fresh abstract values forgets what refinement would recover;
- skipping proposition-valued *calls*, but not the same code outside a call, made a block observe differently sealed and unsealed;
- non-structural recursion closes off on abstract arguments but diverges once they are instantiated.

7.6 Typing and conversion are preserved

Theorem 9 (preservation). For a program accepted by [Def] and [Rec]:

1. a type A of sort s at Ω translates to $\Gamma_\Omega \vdash A^\dagger : s^\dagger$;
2. if $\Omega \vdash A \equiv B$, then $A^\dagger = B^\dagger$ is provable in CIC_L (definitionally in ECIC_L);
3. if $\Omega \vdash t \Downarrow v : A \dashv \Omega'$, then $t^\dagger$ maps views of Ω ’s shape to $A^\dagger$ and views of Ω' ’s shape;
4. a definition $f : \Pi(\bar{x} : \bar{A}).B$ translates to an inhabitant of $(\Pi(\bar{x} : \bar{A}).B)^\dagger$, in the injective part when B is a borrow type.

Proof sketch. By induction on derivations. [Def]’s goal is the [Call-type] of the generic call, which is $B^\dagger(\bar{a})$ by definition. [Split] on an abstract value is dependent elimination on a CIC variable, and applying the refinement to the environment, the goal and every stored type *is* its motive; generalising a sealed program first is CIC ’s generalisation. After a non-tail split, the checked arms define the closed-off block’s function, and the continuation starts from its application (Theorem 2), so nothing is lost. [Rec] makes the translation a strong recursion, and erased terms leave the view unchanged. For conversion, equal normal forms translate equally (Theorem 2), and the **Eq** rules are **propext** instances: $((a, b) = (a', b')) = (a = a' \wedge b = b')$, $(a = a) = \top$, $(\top \wedge P) = P$. **J** becomes Lean’s eliminator, which needs the endpoints **J** carries. Proof irrelevance is sound because *Ochr* propositions translate to CIC propositions. ■

The hardest case is [Call-type]. It uses Theorem 3, and Lemma 4 and Lemma 5, and so owner sets and the injective restriction. It also needs **propext**, [Rec] for the induction hypotheses, and the capture of a Π -type’s free variables at formation, without which a call site’s type is no instance of the type proved.

7.7 Consistency

Corollary 10 (consistency). No closed term has type **Eq Nat Z (S Z)** or $\Pi(P : \text{Prop}). P$. More generally, if $\vdash t : P$ for a closed proposition P , then $P^\dagger$ holds in the set model.

Proof. A closed term has an empty view, so by Theorem 9 it translates to an inhabitant of $P^\dagger$ in ECIC_L . Carneiro’s model interprets ECIC_L in ZFC with one inaccessible cardinal per universe level used, and in that model $(\text{Eq Nat Z (S Z)})^\dagger = \emptyset$. ■

The strong form matters: proofs are never run, so consistency cannot lean on the checker diverging.

7.8 What is mechanised

Mechanised in the accompanying Lean development: *[to be filled in: the theorems proved in ochr/core/meta-lean/, e.g. Theorem 3 (1)–(2), the equations of Theorem 2, Corollary 8, Theorem 6, Lemma 5, for the first-order fragment]*. The fragment has natural numbers, unit, pairs, places, borrows, matching, recursive definitions (including ones returning borrows), opaque definitions for abstract functions, and erased proofs. A computation means its concrete run on Lean data, so the target is Lean itself, and **Id** is Lean’s equality on observation tuples. On paper only: the dependent layer (parts (1), (2) and (4) of Theorem 9 for Π -types over **Id**, universes and [Split]’s motive), Corollary 10, termination of higher-order programs, Theorem 7 for non-ground refinements, and the forcing-model correspondence. The dependent layer is standard given Carneiro’s model. The new part is the effectful layer, and that is where the mechanisation goes.

8 Implementation

We implemented the calculus as an executable reference checker in Lean 4, about 2,200 lines with no dependencies, plus the examples and tests. Its purpose is not performance but fidelity: a hand derivation can be charitable where the rules are silent, and a program cannot. Every example in this paper is checked by it, written in a concrete syntax close to the one used here:

```
ochr E1 {
  def AddM (x : &Nat) (y : Nat) : Unit by x :=
    match *x { Z => *x := y | S p => AddM(&p, y) }

  def AddMZero (x : &Nat) : Id Unit (AddM(x, 0)) () by x :=
    match *x { Z => refl | S p => AddMZero(&p) }

  reject def WriteThenRefl (x : &Nat) : Id Nat (*x) 5 := *x := 5; refl
}
```

A **def** is expected to be accepted and a **reject def** to be rejected, and each verdict is an assertion checked during the build, so a build succeeds only if every verdict is as expected. Each test file also asserts how many verdicts it contains, which guards against a test file that silently checks nothing.

Structure. The machine, closing off, sealed-program normalisation, splitting, closing off stuck blocks, observation, **Id**, [Call-type], [Def] and [Rec] form one mutually recursive block of about a thousand lines. Terms use de Bruijn indices, and the variables introduced by [Close] have fixed names, so “same normal form” is structural equality. The implementation follows the rules closely enough that its trace of **AddMZero** reproduces the derivation of Section 2 symbol for symbol, including the successor that the environment supplies to the induction hypothesis.

Test suite. The suite contains every example of this paper: **AddM**, **AddMZero**, **Add** and **AddZero**; the returned-borrow examples **TailM**, **AddM'** and **AddMEq**; branching with borrows whose origin depends on the branch; opaque function arguments; and the programs of Section 2 that pass a proof about the current, mutated state. It also contains every program that must be rejected, and in particular every false proof found while designing the calculus, each of which is a regression test. At the time of writing all 105 verdicts are as expected, and a clean build of the checker and the suite takes about ten seconds.

Every side condition is load-bearing. Each non-standard rule has a switch that turns it off, and the build also asserts a *counterfactual ledger*: with one rule switched off, exactly the tests that rule is responsible for change their verdict, and no others. Without entry-value recursion (Section 6), the non-terminating “proofs” of Section 6 are accepted. Without exclusive access, a well-typed program writes through an ended borrow. Without running erased terms on a private copy, a proof that writes changes the verdict on programs around it. And without the rule that a recursive function occurs only as the head of a call, a function that passes itself to a helper proves **Eq Nat 0 1**. The ledger has one empty row: switching off owner sets (Section 5) changes no verdict, because every program that would need them requires a closure capturing a borrow, which the core excludes. Owner sets remain in the calculus because the metatheory needs them (Lemma 4).

What running the rules found. Writing the checker exposed three gaps in an earlier version of the rules, each now fixed: the recursive-function-as-a-value loophole above; an argument evaluated earlier in the same call being invalidated by a later one (**f(&x, x)** ends the first borrow while reading the second argument); and recursive calls inside nested closures, which must be checked against the entry values of the enclosing function.

9 Related work

Functional translation of borrows. Aeneas [10] translates safe Rust into a pure functional program in Lean, F* or Coq. Its symbolic semantics of the low-level borrow calculus is the direct ancestor of our machine: we take its values with loans and borrows, its lazy ending of borrows, and its insight that a mutable borrow parameter is a forward computation plus a backward function. Later work gives that symbolic semantics joins and loops

and proves it sound as a borrow checker [11]. The difference is where and when the translation happens. Aeneas translates eagerly, whole-program, once, and the user reasons about the translated artifact, where backward functions are named definitions. Ochr never produces a translated program. The same state-passing content appears lazily inside conversion, only as far as a particular equation needs, and only in source syntax: a sealed program is what Aeneas would call an application of a backward function. Aeneas is modular in that it never unfolds a callee; Ochr must unfold, because unfolding is what definitional equality is, and closing off is what keeps the unfolding finite.

Prophecies. RustHorn [16], RustHornBelt [15] and Creusot [8] model a mutable borrow as its current value paired with a *prophecy* $\hat{x}$ of its final value, resolved when the borrow ends. A filled loan in Ochr is a prophecy that has been resolved by the machine, and the hole in a sealed program for a returned borrow is an unresolved one. Prophecies are logic variables introduced at borrow creation; Ochr introduces no variables at all outside binders, which is what keeps concrete evaluation free of symbols (§Section 7).

Specification-based verification of imperative code. Verus [12], Creusot, Prusti, Flux [13] and VeriFast verify imperative code against specifications written separately, in pre- and postconditions, refinement types or separation logic, and discharge the obligations with automated provers. Hoare type theory [18] and Ynot internalise the triples in a dependent type theory; F*’s Dijkstra monads [2] internalise weakest preconditions and let **reify** turn a stateful computation into a state-passing function. In all of these the effectful program is specified, not compared: two programs with the same specification are not thereby equal. In Ochr the specification of an in-place function is another program, or a theorem about the function itself, and equality of computations is a type.

Effects in dependent type theory. The fire triangle [20] shows that substitution, dependent elimination and observable effects cannot coexist consistently, and ∂ CBPV organises the possible compromises. Ochr sits in the call-by-value corner: dependent elimination is kept, and substitution into types is restricted to values, where a computation in a type contributes only its observation. The observation of a computation is *thunkable* in their sense, and ownership gives a syntactic criterion for thunkability, which they note is otherwise hard to capture intensionally. Ahman’s eMLTT [3] equips computations with definitional equations from an algebraic theory of global state; Ochr’s state is structured, local and borrow-shaped, and its equations come from running the program rather than from an axiomatised theory.

Observational equality. Observational type theory [4, 19] computes the equality type by the structure of the type: equality of pairs is a pair of equalities, equality of functions is pointwise. Ochr’s **Id** between computations is an instance of this idea at the type of observations, and we adopt the observational computation rules for the data types of the core.

Pure languages with in-place updates. Lean 4 performs destructive updates on values it can prove unshared at runtime [21], and Koka’s fully in-place fragment [14] guarantees statically that a function allocates nothing. Programs in these languages are pure, so their proofs are ordinary, but the programmer writes functional code and cannot express a borrow that is returned to the caller. Linear and quantitative type theories [5] and ATS [23] track resources in types; Ochr uses the same separation between runtime usage and erased type-level usage as quantitative type theory, where a variable used in a type is not consumed.

Relational reasoning. ReLoC [9] and relational Hoare logics prove two programs equivalent inside a program logic. Ochr’s **Id** is a relational statement about two computations, but it is decided, where possible, by conversion, and it lives in the same type theory as the programs.

10 Discussion

10.1 What generalises directly

Inductive types. The core has only **Nat** and **Unit**, but nothing in the rules is specific to them. A constructor with several fields gives several sub-places **p.1**, ..., **p.k**; **[Split]** refines an abstract value to a constructor applied

to fresh abstract values; [Rec] asks for a strict subterm of the entry value. In-place list append is **AddM** with **Cons** in place of **S**, and its theorems are proved the same way.

Opaque definitions. A definition may be declared with a type and no body, or used through an abstract function value. A call to it is stuck at once and closed off, so its effects are recorded as sealed programs mentioning it, and every theorem about its callers remains available. This is how Ochr recovers the modularity of Aeneas’s symbolic execution [10], which never unfolds a callee: the programmer chooses, per definition, whether conversion may look inside.

Shared borrows. Immutable borrows are simpler than mutable ones in this setting: a shared borrow never has a final value different from its initial one, so closing off a call that takes one needs no sealed program for it. We omitted them to keep the core small.

10.2 The frontier

Borrows stored in data. The central restriction of the core is that borrow types never occur inside other types. It rules out an iterator that returns a list of borrows into a list, $\text{IterM} : \&(\text{List Nat}) \rightarrow \text{List } (\&\text{Nat})$. There, the number of borrows returned is itself unknown on symbolic input, so no fixed set of holes can stand for their final values. The natural generalisation is a hole that stands for a whole data structure of final values, filled when the last borrow in the structure ends, which is the neutral form of an Aeneas region abstraction over a symbolic value with a borrow projector. The same restriction excludes closures that capture borrows.

Loops. A loop is a tail-recursive local function, and its type is its invariant. Closing off a loop that is stuck on an abstract value is closing off that function. We have not designed the surface form, nor checked whether the invariants that arise are the ones programmers expect to write.

Interior mutability and unsafe code. Closing off is sound because a call can affect only what it is passed. Global state, interior mutability or unsafe aliasing would each break this frame property, and with it the rule. They would need to be confined, as Rust confines unsafe code behind safe interfaces, and given sealed-program semantics through their interfaces.

10.3 Costs

Checking cost. Type checking runs programs. On concrete data this is the cost of running them; on symbolic data each stuck call is closed off once, and each refinement re-runs the sealed programs that mention the refined value. Sealed programs are closed and canonical, so their normal forms can be cached and shared, and a refinement only re-runs what mentions it. Our implementation does no caching; **TODO: timings from the implementation**. We have not measured large programs.

Copying. The calculus copies borrow-free data on reads. A compiled program should move instead, and an affine usage discipline on runtime code, outside the core, licenses implementing each last-use copy as a move. Types are exempt from that discipline, which is what lets $x + x = 2 \cdot x$ be stated for a type without a copy operation; this is the same separation between runtime usage and erased usage as quantitative type theory [5].

What the programmer sees. Goals and hypotheses are propositions about values, and every value that appears in them is either data, an abstract value named after the parameter it came from, or a sealed program, which is source code. A sealed program can be printed as the user’s own wrapper when one exists ($\text{flet } c = \sigma; \text{AddM}(\&c, \theta); c \mid \text{is Add}(x, \theta)$). Nothing in a goal mentions loans, borrows, environments or backward functions. We regard this as a design invariant, and it is a useful test for extensions: a feature whose goals cannot be printed as source code is one the programmer would need a new concept to reason about.

11 Conclusion

The usual way to verify an efficient imperative program is to write it twice. We have shown that, for programs whose mutation is disciplined by Rust-style ownership, the second copy can be generated on demand, inside the type checker, as source code: when definitional equality meets a call it cannot run, it closes the call off into sealed programs that own their state, and a later case split runs them again. Combined with an observational equality

that compares computations by their results and their writes, this lets theorems about in-place code be stated about that code and proved by the same structural recursion one would use for pure code. Often the in-place proof is the simpler one, because the borrow structure of the environment performs the congruence steps a pure proof must spell out.

The core calculus is small, and every side condition in it is there because a smaller one admitted a proof of false. The main open direction is borrows stored inside data, where the neutral form of a stuck call must describe a whole structure of final values rather than a fixed set of places.

Bibliography

- [1] Andreas Abel. 2013. Normalization by Evaluation: Dependent Types and Impredicativity. In *Habilitation thesis, LMU Munich*, 2013.
- [2] Danel Ahman, Catalin Hritcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *POPL*, 2017.
- [3] Danel Ahman. 2018. Handling Fibred Algebraic Effects. In *Proc. ACM Program. Lang. (POPL)*, 2018.
- [4] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational Equality, Now!. In *PLPV*, 2007.
- [5] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *LICS*, 2018.
- [6] William J. Bowman, Youyou Cong, Nick Rioux, and Amal Ahmed. 2018. Type-Preserving CPS Translation of Σ and Π Types is Not Not Possible. In *Proc. ACM Program. Lang. (POPL)*, 2018.
- [7] Mario Carneiro. 2019. The Type Theory of Lean.
- [8] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*, 2022.
- [9] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *LICS*, 2018.
- [10] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proc. ACM Program. Lang. (ICFP)*, 2022. 116:1–116:31.
- [11] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. 2024. Sound Borrow-Checking for Rust via Symbolic Semantics. In *Proc. ACM Program. Lang. (ICFP)*, 2024.
- [12] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023.
- [13] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. In *PLDI*, 2023.
- [14] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. 2023. FP^2 : Fully in-Place Functional Programming. In *Proc. ACM Program. Lang. (ICFP)*, 2023.
- [15] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI*, 2022.
- [16] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *ESOP*, 2020.
- [17] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.
- [18] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and Separation in Hoare Type Theory. In *ICFP*, 2006.
- [19] Loïc Pujet and Nicolas Tabareau. 2022. Observational Equality: Now for Good. *Proc. ACM Program. Lang. (POPL)* 6, (2022).

- [20] Pierre-Marie Pédrot and Nicolas Tabareau. 2020. The Fire Triangle: How to Mix Substitution, Dependent Elimination, and Effects. *Proc. ACM Program. Lang. (POPL)* 4, (2020), 58:1–58:28.
- [21] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *IFL*, 2019.
- [22] Théo Winterhalter, Matthieu Sozeau, and Nicolas Tabareau. 2019. Eliminating Reflection from Type Theory. In *CPP*, 2019.
- [23] Hongwei Xi. 2003. Applied Type System (extended abstract). In *TYPES*, 2003.